Ex. No. 08	Bridging the Divide: Seamless Inter-Process Communication with pipes
Date: 10.03.2025	

Objective:

To understand and analyze the concept Seamless Inter-Process Communication with pipes in Unix/Linux.

Theme: Online Learning Tracker

Description:

Pipes enable efficient data exchange between processes by creating a direct communication channel. They allow one process to send information while another reads it, ensuring synchronized execution. Unnamed pipes facilitate communication between related processes, while named pipes (FIFOs) extend this capability to unrelated processes. By leveraging pipes, applications can achieve real-time data transfer and improved modularity. This mechanism is widely used in process synchronization, file handling, and system-level operations.

Exercise:

Question 1: Parent-Child Communication Using Unnamed Pipes.



```
┌──(kali㉿kali)-[~]
└─$ ./parent_child_communication
Child received feedback: Customer Feedback: Service was excellent!
```

```
kali% ./exp101
Child received progress data: Student: Mercy, Course: OS, Progress: 85%
kali%
```

Question 2: Named Pipe (FIFO).

```
 GNU nano 6.0                                    exp102.c *
// q2_named_pipe_fifo.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>

int main() {
    char *fifo = "progress_fifo";
    mkfifo(fifo, 0666); // Create FIFO if not exists

    if (fork()) {
        int fd = open(fifo, O_WRONLY);
        char *msg = "Update: Mercy completed Module 2 of DBMS";
        write(fd, msg, strlen(msg) + 1);
        close(fd);
    } else {
        char buffer[100];
        int fd = open(fifo, O_RDONLY);
        read(fd, buffer, sizeof(buffer));
        printf("Received via FIFO: %s\n", buffer);
        close(fd);
    }

    unlink(fifo);
    return 0;
}
```

```
kali% gcc exp102.c -o exp102 -lrt
kali% ./exp102
Received via FIFO: Update: Mercy completed Module 2 of DBMS
kali%
```

Question 3: Splitting Large Data Using Pipes.

```
 GNU nano 6.0                                    exp103.c *
// q3_split_file_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>

int main() {
    int pipefd[2];
    char *filename = "progress_log.txt";

    // Create file if it doesn't exist
    FILE *fp = fopen(filename, "w");
    fprintf(fp, "Student: Mercy, Modules Completed: 5\n");
    fclose(fp);

    pipe(pipefd);
    if (fork() == 0) {
        // Child reads from file, sends through pipe
        close(pipefd[0]);
        FILE *readfp = fopen(filename, "r");
        char line[100];
        while (fgets(line, sizeof(line), readfp)) {
            write(pipefd[1], line, strlen(line));
        }
        fclose(readfp);
        close(pipefd[1]);
    } else {
        // Parent receives from pipe
        close(pipefd[1]);
        char buffer[100];
        printf("Parent received chunks:\n");
        while (read(pipefd[0], buffer, sizeof(buffer)) > 0) {
            printf("%s", buffer);
        }
        close(pipefd[0]);
    }

    return 0;
}
```

```
┌──(kali㉿kali)-[~]
└─$ ./splitting_feedback
Child received feedback chunk: Customer says: Good service!
```

```
kali% gcc exp103.c -o exp103 -lrt
kali% ./exp103
Parent received chunks:
Cataloging courses...
Tracking progress...
Marking courses as completed...
Completion updated!
Courses cataloged!
Progress tracked!
Cataloging courses...
Tracking progress...
Marking courses as completed...
Completion updated!
Courses cataloged!
Progress tracked!
Student: Mercy, Modules Completed: 5
ed...
Completion updated!
Courses cataloged!
Progress tracked!
Student: Mercy, Modules Completed: 5%
kali%
```

Question 4: Producer-Consumer Model Using Pipes.

```
 GNU nano 6.0                                        exp104.c *
// q4_producer_consumer_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int pipefd[2];
    pipe(pipefd);

    if (fork() == 0) {
        // Consumer
        close(pipefd[1]);
        char buffer[100];
        read(pipefd[0], buffer, sizeof(buffer));
        printf("Consumer received: %s\n", buffer);
        close(pipefd[0]);
    } else {
        // Producer
        close(pipefd[0]);
        char *data = "Mercy completed AI Quiz with 92%";
        write(pipefd[1], data, strlen(data) + 1);
        close(pipefd[1]);
    }

    return 0;
}
```

```
kali% gcc exp104.c -o exp104 -lrt
kali% ./exp104
kali% Consumer received: Mercy completed AI Quiz with 92%
```

Question 5: Transmitting Theme-Related Commands Using Pipes

```
 GNU nano 6.0                                  exp103.c *
// q3_split_file_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>

int main() {
    int pipefd[2];
    char *filename = "progress_log.txt";

    // Create file if it doesn't exist
    FILE *fp = fopen(filename, "a+");
    fprintf(fp, "Student: Mercy, Modules Completed: 5\n");
    fclose(fp);

    pipe(pipefd);
    if (fork() == 0) {
        // Child reads from file, sends through pipe
        close(pipefd[0]);
        FILE *readfp = fopen(filename, "r");
        char line[100];
        while (fgets(line, sizeof(line), readfp)) {
            write(pipefd[1], line, strlen(line));
        }
        fclose(readfp);
        close(pipefd[1]);
    } else {
        // Parent receives from pipe
        close(pipefd[1]);
        char buffer[100];
        printf("Parent received chunks:\n");
        while (read(pipefd[0], buffer, sizeof(buffer)) > 0) {
            printf("%s", buffer);
        }
        close(pipefd[0]);
    }

    return 0;
}
```

```
kali% ./exp105
Executing: echo 'Logging: Mercy started Data Structures'
kali% Logging: Mercy started Data Structures

kali%
```

Question 6: Encrypt & Send data Using Pipes.

```
 GNU nano 6.0                                  exp106.c *
// q6_encrypt_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

void encrypt(char *str) {
    for (int i = 0; str[i]; i++)
        str[i] += 1;  // Caesar cipher (+1)
}

void decrypt(char *str) {
    for (int i = 0; str[i]; i++)
        str[i] -= 1;
}

int main() {
    int pipefd[2];
    pipe(pipefd);

    if (fork() == 0) {
        close(pipefd[1]);
        char encrypted[100];
        read(pipefd[0], encrypted, sizeof(encrypted));
        decrypt(encrypted);
        printf("Decrypted: %s\n", encrypted);
        close(pipefd[0]);
    } else {
        close(pipefd[0]);
        char data[] = "Mercy's progress in ML is 70%";
        encrypt(data);
        write(pipefd[1], data, strlen(data) + 1);
        close(pipefd[1]);
    }

    return 0;
}
```

```
kali% nano exp106.c
kali% gcc exp106.c -o exp106 -lrt
kali% ./exp106
kali% Decrypted: Mercy's progress in ML is 70%
```

Question 7: Simulating File Transfer Using Pipes.

```
 GNU nano 6.0                                        exp107.c *
// q7_file_transfer_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main() {
    int pipefd[2];
    char *file = "learning_status.txt";
    FILE *fp = fopen(file, "a+");
    fprintf(fp, "Mercy - Web Dev - Completed\n");
    fclose(fp);

    pipe(pipefd);

    if (fork() == 0) {
        close(pipefd[1]);
        char buffer[100];
        read(pipefd[0], buffer, sizeof(buffer));
        printf("Received file content: %s\n", buffer);
        close(pipefd[0]);
    } else {
        close(pipefd[0]);
        FILE *sendfp = fopen(file, "r");
        char line[100];
        fgets(line, sizeof(line), sendfp);
        write(pipefd[1], line, strlen(line) + 1);
        fclose(sendfp);
        close(pipefd[1]);
    }

    return 0;
}
```

```
kali% nano exp107.c
kali% gcc exp107.c -o exp107 -lrt
kali% ./exp107
Received file content: Mercy - Web Dev - Completed
```

Question 8: Sorting Data Using Pipes.

```
kali% gcc exp108.c -o exp108 -lrt
kali% ./exp108
Sorted Modules: AI, CN, DBMS, ML, OS
kali%
```

```
 GNU nano 6.0                                  exp108.c *
// q8_sort_pass_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int compare(const void *a, const void *b) {
    return strcmp(*(char **)a, *(char **)b);
}

int main() {
    int pipefd[2];
    pipe(pipefd);

    if (fork() == 0) {
        close(pipefd[1]);
        char sorted[200];
        read(pipefd[0], sorted, sizeof(sorted));
        printf("Sorted Modules: %s\n", sorted);
        close(pipefd[0]);
    } else {
        close(pipefd[0]);
        char *modules[] = {"AI", "DBMS", "OS", "ML", "CN"};
        int n = 5;
        qsort(modules, n, sizeof(char *), compare);

        char combined[200] = "";
        for (int i = 0; i < n; i++) {
            strcat(combined, modules[i]);
            if (i < n - 1) strcat(combined, ", ");
        }

        write(pipefd[1], combined, strlen(combined) + 1);
        close(pipefd[1]);
    }

    return 0;
}
```

Question 9: Counting Records Using Pipes.

```
 GNU nano 6.0                                  exp109.c *
// q9_count_records_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int pipefd[2];
    FILE *fp = fopen("progress_data.txt", "w");
    fprintf(fp, "Mercy\nZiyi\nMinju\nLouis\n");
    fclose(fp);

    pipe(pipefd);

    if (fork() == 0) {
        close(pipefd[1]);
        int count;
        read(pipefd[0], &count, sizeof(int));
        printf("Total Students Tracked: %d\n", count);
        close(pipefd[0]);
    } else {
        close(pipefd[0]);
        FILE *fp = fopen("progress_data.txt", "r");
        int count = 0;
        char name[50];
        while (fgets(name, sizeof(name), fp)) count++;
        fclose(fp);
        write(pipefd[1], &count, sizeof(int));
        close(pipefd[1]);
    }

    return 0;
}
```

```
kali% gcc exp109.c -o exp109 -lrt
kali% ./exp109
Total Students Tracked: 4
kali%
```

Question 10: Bidirectional Communication Using Pipes.

```
 GNU nano 6.0                                  exp1010.c *
// q10_bidirectional_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int p1[2], p2[2];
    pipe(p1); pipe(p2);

    if (fork() == 0) {
        // Child process
        close(p1[1]); close(p2[0]);
        char msg[100];
        read(p1[0], msg, sizeof(msg));
        printf("Child got: %s\n", msg);
        char reply[] = "Update received. Great job, Mercy!";
        write(p2[1], reply, strlen(reply) + 1);
        close(p1[0]); close(p2[1]);
    } else {
        // Parent process
        close(p1[0]); close(p2[1]);
        char *msg = "New progress: Mercy completed CN!";
        write(p1[1], msg, strlen(msg) + 1);
        char response[100];
        read(p2[0], response, sizeof(response));
        printf("Parent received: %s\n", response);
        close(p1[1]); close(p2[0]);
    }

    return 0;
}
```

```
kali% ./exp1010
Child got: New progress: Mercy completed CN!
Parent received: Update received. Great job, Mercy!
kali%
```

Result:

The concept of Seamless Inter-Process Communication with pipes in Linux and Unix has been done successfully.